

【实验报告】

课程名： 计算机组成与体系结构（H）

实验名称： lab5 MMU

学生： 杨箫羽

学号： 24300240069

时间： 2026 年 5 月 26 日

一、实验内容与实现思路说明

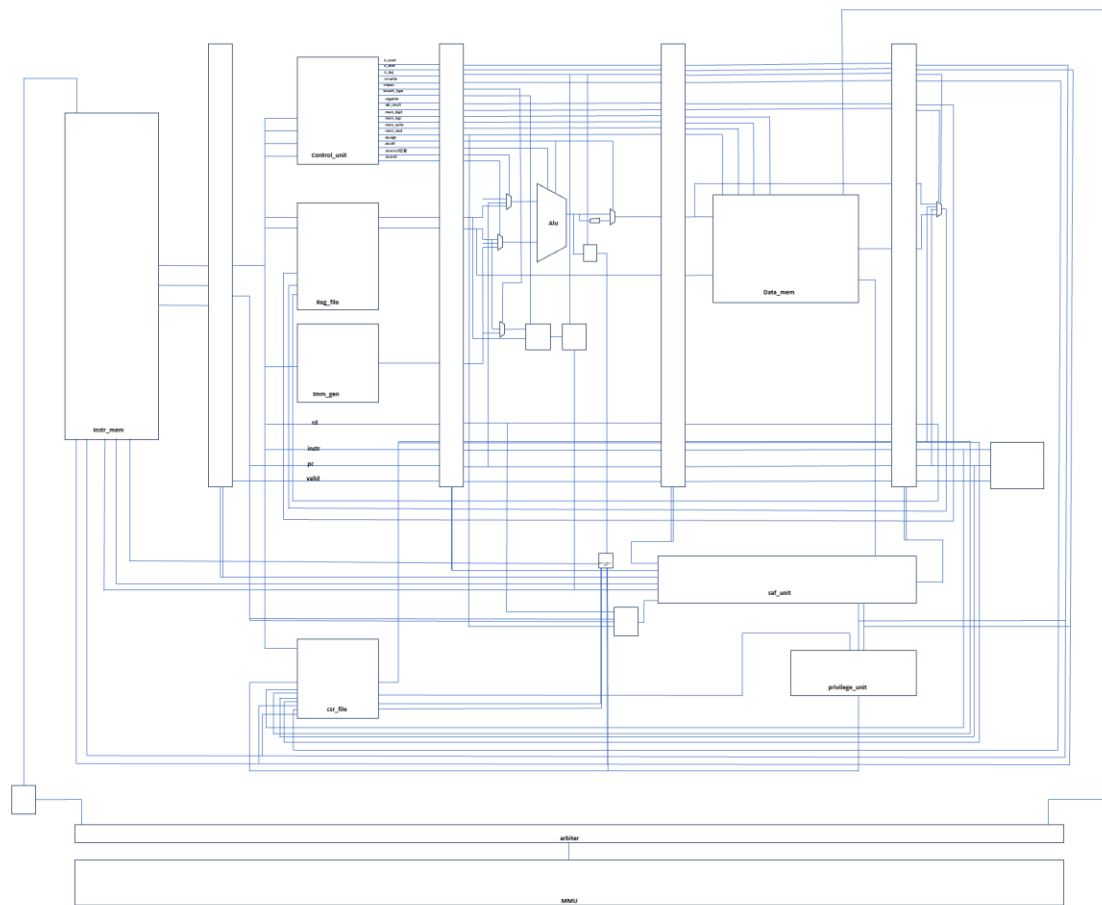
实验 5 要求实现 mret 和 ecall 指令以及 Sv39 虚拟内存机制。为了实现这些指令，我增加了 privilege mode unit 来维护特权状态，增加了 is_ecall 和 is_mret 两个控制信号。为了实现 MMU，我利用方式 1 的思路，增加了 mmu 模块、dbus_arbiter 模块，ibus_to_dbus 模块，先将 instrmem 的 ibus 转换成 dbus，然后进行仲裁，最后通过 mmu 模块页表进行 dbus 的地址翻译，得到物理地址的 dbus 后连接外设。通过 verilog 仿真和上板检验。

下面分总体电路图、新增的模块讲解和做出重要修改的模块讲解三部分进行具体说明。

```
TEST= ./build/emu --diff /root/26-Arch/ready-to-run/riscv64-nemu-interpretor-so -i ./ready-to-run/lab5/kernel.bin || true
Emu compiled at Mar 11 2026, 14:50:48
The image is ./ready-to-run/lab5/kernel.bin
Using simulated 256MB RAM
Using /root/26-Arch/ready-to-run/riscv64-nemu-interpretor-so for difftest
[src/device/io/mmio.c:18,add_mmio_map] Add mmio map 'clint' at [0xa2000000, 0xa200ffff]
[src/device/io/mmio.c:18,add_mmio_map] Add mmio map 'uartlite' at [0x40600000, 0x4060000c]
[src/device/io/mmio.c:18,add_mmio_map] Add mmio map 'uartlite1' at [0x23333000, 0x2333300f]
The first instruction of core 0 has committed. Difftest enabled.
xv6 kernel is booting
[WARNING] difftest store queue overflow
kinit ok
procinit ok
trapinit ok
plicinit ok
userinit ok
Return from init! Test passed
```



二、总体电路图



三、修改和新增的重要模块

1、privilege_unit

接口：

```
module privilege_unit (  
    input logic clk,  
    input logic rst,  
    input logic is_mret,  
    input logic is_ecall,  
    input logic [1:0] mpp,  
    output logic [1:0] privil_mode  
);
```

功能：privilege_unit 模块用于维护 CPU 当前特权级。复位后默认进入 M 模式；执行 mret 时，根据 CSR 中的 mpp 恢复特权级；执行 ecall 时，切换至 M 模式进入异常处理流程。

2、MMU

```
module mmu import common::*;(
    input  logic clk,
    input  logic reset,
    input  logic flush,

    input  dbus_req_t  cpu_req,
    output dbus_resp_t cpu_resp,

    output dbus_req_t  mem_req,
    input  dbus_resp_t mem_resp,

    input  logic [1:0]  privil_mode,
    input  logic [63:0] satp

```

接口：);

功能：MMU 模块基于 Sv39 实现三级页表遍历，通过状态机逐级读取页表项，并根据 SATP 与当前特权级判断是否开启地址翻译，最终完成虚拟地址到物理地址的转换。

3、final_redirect_pc_unit

目的：为了在 mret 和 ecall 指令进行 pc 修改的时候服用 lab3 的 branch 分支跳转，增加一个 unit 进行重定位 pc 的选择。

```
module final_redirect_pc_unit(
    input logic [63:0] branch_redirect_pc,
    input logic [63:0] csr_mepc,
    input logic [63:0] csr_mtvec,
    input logic is_ecall,
    input logic is_mret,
    output logic [63:0] final_redirect_pc
);
```

接口：

功能：如果是 ecall，输出 mtvec，如果是 mret，输出 mepc，否则输出 branch_redirect_pc

4、dbus_arbiter

接口：

```
module dbus_arbiter import common::*; #(
    parameter int NUM_INPUTS = 2,
    localparam int MAX_INDEX = NUM_INPUTS - 1
) (
    input  logic clk,
    input  logic reset,

    input  dbus_req_t  [MAX_INDEX:0] reqs,
    output dbus_resp_t [MAX_INDEX:0] resps,

    output dbus_req_t  final_req,
    input  dbus_resp_t final_resp
);
```

功能：模仿 util 的仲裁器，dbus_arbiter 模块用于仲裁多个数据总线请求，通过轮询选择有效请求发送至共享总线，并在访存完成后将响应返回对应主设备，实现访存请求的串行化管理。

5、control_unit 新增信号和指令情况

		ecall	mret
opcode		1110011	1110011
funct3		0	0
funct7			
bit30			
immediate		0	302
alusign	1时针对rv64i这种指令，做32->64的扩展	0	0
aluctrl	(0-9) +/异或/或/与/-/有符号小于/无符号小于	0	0
alusrca	0为0, 1为rs1val,2为pc	0	0
alusrcb	0为rs2val, 1为imm, 3为0	0	0
regwrite	0不写, 1写	0	0
mem_write	0不写, 1写	0	0
mem_read	0不读, 1读	0	0
mem_sign	0时符号扩展, 1时零扩展	0	0
mem_digit	8/16/32/64	0	0
wb_result	0为alu结果, 1为mem读结果, 2为pc+4, 3为pc+imm	0	0
branch_type	0-5分别代表=, !=, 有符号<,有符号>=, 有符号<=,有符号>	0	0
cmpsrc	0为rs2val, 1为imm	0	0
is_baj	0代表都不是, 1代表b类, 2代表jal,3代表jalr	0	0
csrwrite	0不写, 1写	0	0
is_ecall	1是ecall	1	0
is_mret	1是mret	0	1

四、收获

因为我是在自己写完代码之后用 AI 帮忙看错在哪里，加一些补丁，这种情况下如果没有把 AI 修改的逻辑和原有的逻辑完全理顺的话，加上如果没有很好的变量含义注释导致 AI 不能完全理解原有代码，会导致代码逻辑变得很复杂，其实就是多了很多中间信号（我觉得增加信号是解决 sv 代码复杂逻辑的一个方法，但是如果定义不良就会写出质量不高、难以理解的代码），他们的定义语义模糊（而且自然语言本身就容易描述不准确），相互重叠或者前后不一致。之前 lab1、2 写 instrmem 和 datamem 遗留了一些类似的问题，加上这次 mmu 和 dbus_arbiter 的新增，就导致访问内存的 bug 比较多。

最后代码逻辑基本正确，但是仿真一直输出两倍字符 tteesstt ppaasssseedd。解决方法是让 AI 帮忙重新理顺变量的确切含义，把冗余的含义去掉，理顺之后就可以正常仿真了。